Historical Retention and Runtime Costs in a Learned Priority Queue

Sahil Siddiqui

A released-artifact study with controlled ablations and two-host evaluation

Abstract

A learned priority queue must do more than locate the next key: it must maintain prediction indexes, release removed items, and pass results efficiently to the application. We audit a released learning-augmented priority queue and find that a dictionary retains historical priorities after extraction. In a bounded-live test on the Mac, deleting a mapping when its final queued copy is removed reduces drained-queue traced memory from 35.18 to 0.024 MiB while preserving the tested search behavior. The ownership effect repeats on a second physical laptop. In a separately implemented native evaluation, the direct binary heap has the lowest median on 7/7 workloads on both hosts, while smaller index and shared-interface effects vary by configuration. This provides a reproducible case where historical object ownership dominates retained storage and where optimizing an auxiliary index alone does not determine end-to-end performance. The maintainer remedy is a narrow dictionary cleanup that leaves hint links intact.

1. Introduction

Predictions can reduce the number of comparisons needed to locate a key in an ordered structure. The implementation must nevertheless evaluate the prediction, maintain an auxiliary index, and manage the objects used by the queue. For inexpensive numeric comparisons, these costs may materially affect elapsed time. Priority-queue engineering has long studied such differences between abstract operation costs and realized performance [3]. This paper asks how those issues appear in a specific, public learned-priority-queue artifact.

Benomar and Coester introduce priority queues with pointer, dirty-comparison, and rank predictions [1]. Their empirical evaluation reports comparison counts. We extend that evaluation to runtime and memory on a pinned released implementation. We separate an index substitution that preserves queue behavior from a larger repair that changes bookkeeping and duplicate handling.

The central observation is a mismatch between the number of live entries and the amount of retained metadata. The released skip list records each newly encountered key in a dictionary, but extraction does not remove the corresponding dictionary entry. We characterize this mechanism with a bounded-live synthetic workload, measure its impact in graph searches, and evaluate changes that keep one node per insertion separately from a combined repair. A separately implemented C++ evaluation then tests whether the broad runtime result persists when the Python versus C-extension difference is removed.

The contribution is a reproducible empirical analysis of this artifact: a demonstrated retention mechanism, tests of cleanup that keeps one node per insertion, explicit accounting for training and query memory, and native controls that distinguish queue costs from the cost of a common distance-to-vertex routing map.

Algorithms with predictions seek gains from useful advice while retaining guarantees when predictions are inaccurate. Lykouris and Vassilvitskii [10] develop this approach for caching, obtaining prediction-sensitive competitive performance with a worst-case cap. Mitzenmacher and Vassilvitskii [11] place this objective within beyond-worst-case algorithm design. Benomar and Coester [1] apply prediction-based analysis to priority queues through pointer, dirty-comparison, and rank advice. Our study examines the implementation costs of that advice interface: predictor evaluation, historical ownership, and the bookkeeping needed to preserve insertion hints during cleanup.

Related empirical work establishes the need for practical-cost controls. Larkin, Sen, and Tarjan [6] evaluate priority queues on synthetic, road, and simulation workloads, examining cache behavior and operation-sequence sensitivity. For learned indexes, SOSD [7] compares lookup latency with optimized conventional structures and measures

construction and hardware costs. Liu et al. [8] analyze memory-bound internal search in PGM indexes, while Bellomo et al. [9] jointly evaluate lookup, construction, and memory across learned, compressed, and traditional indexes. Those static-index studies motivate our implementation-cost controls. Our narrower question concerns historical node ownership and the hint dependencies that constrain its repair in a pinned public queue.

2. Artifact, variants, and scope

The original repository is pinned to commit 3915cc261e87949b2b0aa724673c2227492d1b08 [2]. The retrieval process appended one terminal newline to three source files; the input manifest records both original and normalized hashes. Original class definitions are loaded through the Python AST. Timing variants remove only analytic comparison-counter assignments and increments. They do not change search branches. Differential tests check that the index-only variants preserve values, node heights, and comparison counts under matched seeds.

Appendix C.1 [1] describes collecting equal-priority items behind one ordered key. The released OnlineSL implementation does not implement the grouping described in Appendix C.1. The pinned OnlineSL code [2] allocates a skip-list node on every successful insertion, including repeated priorities: OnlineSL.insert calls sl.insertES, and insertNextTo samples a height and creates a new Node. Its rankVal/valRank occurrence lists and Dijkstra distance-to-vertex routing buckets do not suppress these node insertions. We call this one-node-per-insertion policy ungrouped. Original/index-only variants and the four ungrouped variants preserve this released policy; compact and native grouped controls are labeled separately.

Variant	Implementation change
Original	Released rank index based on SortedList; historical node dictionary and collision lists remain.
Bitmap	Replace only the predecessor index with a hierarchy of 64-bit bitmaps.
Sparse vEB	Replace only the index with recursive universe decomposition, lazy dictionary clusters, and bitmap leaves.
Compact	Bitmap index plus duplicate-key grouping, direct bucket removal, live-only node mappings, and detached expired links.
Controls	Binary heap, direct SortedList, plain skip list, and an integer monotone radix heap where eligible.

Table 1. Python variants. Compact combines changes to duplicate representation, node ownership, and bucket removal.

The bitmap index visits O(log-base-64 U) levels for universe size U. The sparse vEB index uses recursive universe decomposition with expected-cost Python hash operations [4]. The radix control is a conventional monotone integer queue [5] used with lazy Dijkstra insertions.

Every Python graph kernel uses the same distance-key to LIFO-vertex routing map and removes empty buckets. The specialized heap baseline invokes C-backed heapq operations directly inside a Python kernel. A second heap passes through the shared queue wrapper, measuring dispatch overhead. This keeps queue-level comparisons controlled while leaving open whether the common routing map is a competitive application interface. The native study explicitly addresses that question.

Original/index-only variants are evaluated with deterministic monotone advisors: an exact training-key CDF lookup, a linear approximation, and a 64-bin histogram. The compact queue also changes hint history and random-height consumption when duplicate priorities are grouped. Earlier arbitrary-advice tests exposed failures involving nonmonotone predictions and duplicates; those raw records are preserved separately.

3. Experimental method

The eight base instances comprise the released Brussels road graph, the DIMACS New York road graph, and two sizes each of integer grids, geometric graphs, and sparse random graphs. Brussels contains 2,268 vertices and uses 4,378 key-0 neighbor arcs from 4,393 raw arcs; its size differs from the published figure. New York contains 264,346 vertices and 733,846 arcs, retaining all parallel arcs, and is distinct from the original paper's OSMnx New York input.

Synthetic sizes are 2,025/8,100 for grids and 2,048/8,192 for the geometric and sparse families. Geometric graphs use symmetrized exact eight-nearest-neighbor edges. Sparse graphs include a bidirectional ring and outgoing shortcuts. Ordered adjacency hashes, seeds, reference distances, and source IDs are stored in the artifact. Nine controlled targets scale edge weights by 0.25 or 4, or apply a lopsided two-regime weighting. Their advisors are trained on the untransformed base. These transformations reuse the base graph topology.

Each graph uses three training sources and one predefined near and far held-out query per training source. Each of these six query/model cases pairs a held-out query source with its fitted training model; repeated runs are collapsed to one estimate per case. A training source and its near/far queries form one bootstrap resampling group. Query references are used for source selection and correctness validation, never as predictor inputs. Small base graphs use five paired repetitions; New York and transformed targets use three. The main Python runtime experiment has 390 paired blocks and 4,872 timed method records. Each block runs in a fresh process, shuffles method order, performs a validated method-specific warmup, and measures fresh queue execution. Model training and construction are measured separately. All required advisor objects remain resident during a paired timing block. Across the main Python study, bookkeeping ablation, and native runs on the two hosts, the total is 4,872 + 540 + 3,444 + 3,444 = 12,300 timed graph queries.

Python query timing disables cyclic garbage collection. Forced cleanup is recorded separately. Both wall and process CPU clocks are retained. We summarize per-source medians of within-block paired ratios, then take graph-level medians. Exploratory percentile intervals resample training-source clusters while preserving the near/far composition. The intervals condition on the selected graph and source cases.

Correctness for timed graph runs requires exact full distance-vector agreement with an independent heapq oracle and a drained queue. Untimed checked wrappers verify insertion/extraction counts. Source hashes are recorded at worker launch and completion. All 390 planned main Python timing blocks match their specifications and share one executed-source hash set. Of the 4,872 timings, 46 have wall/CPU ratios above 1.10 and 12 above 1.25; no timing is excluded using those thresholds.

The first host is an Apple M1 MacBook Air with 8 GiB RAM, macOS 14.3.1, and Python 3.13.3. CPU placement, frequency, thermal state, and other host activity are not controlled. The compiled study uses Apple Clang 15, C++17, -O3, and -DNDEBUG without fast-math. Local timing experiments, memory tracing, compilation, and plotting run sequentially.

4. Historical retention and memory

Along a successful execution with finite numeric priorities, let H be the number of distinct priorities inserted during the lifetime of an original queue. The released implementation assigns each newly encountered priority an entry in sl.nodes and never erases that entry during deletion. Starting from an empty map, induction on insertions and extractions gives H map entries after the queue drains. Consequently, this metadata has an Omega(H) entry-count lower bound even when the maximum number of simultaneously live queue entries is fixed.

We test the mechanism independently of graph size with unique increasing priorities, constant rank prediction zero, index universe one, and at most 128 simultaneously live entries. Each method and insertion count runs in a fresh process. Tracing starts after imports and class warmup. After all entries are extracted, full garbage collection occurs while the empty queue object remains alive. Statistics bookkeeping contributes a small amount to traced allocations. All 12 method/length runs return the exact expected extraction sequence.

Total insertions	Original MiB	Bitmap MiB	Compact MiB
1,000	0.344	0.347	0.021
10,000	3.306	3.309	0.021
100,000	35.182	35.186	0.021

Table 2. Traced Python memory after draining and collecting garbage, with the empty queue still alive. The live-entry maximum is 128 in every case.

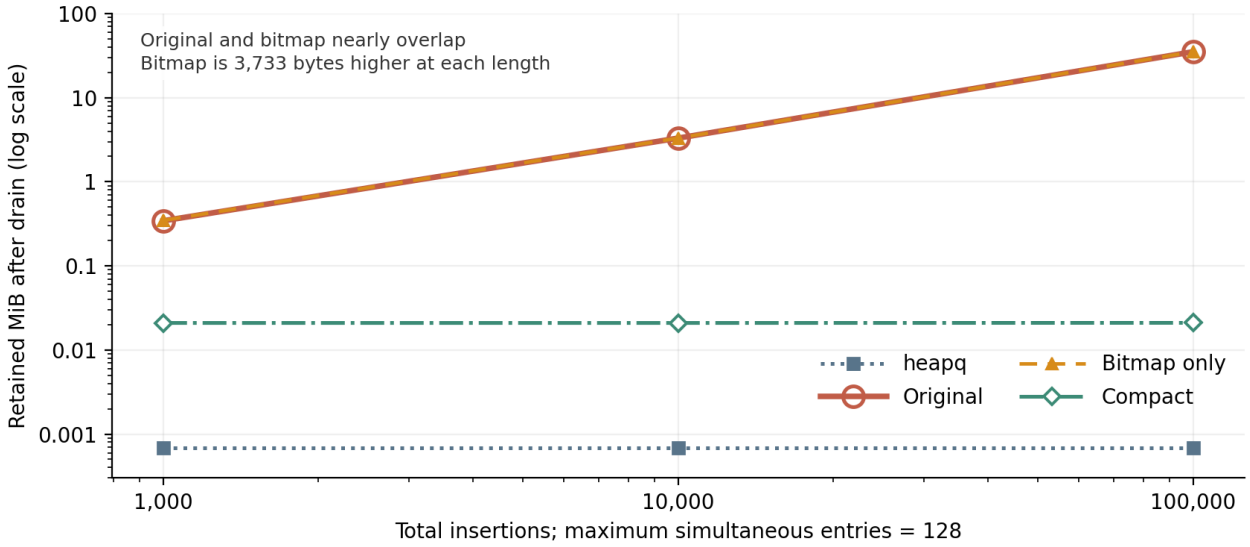


Figure 1. Historical insertions drive retained memory in both original and index-only bitmap variants. Large hollow circles and small filled triangles distinguish their nearly overlapping measurements; coordinates are unchanged. The compact repair removes historical node mappings and is nearly flat over the tested lengths.

At 100,000 insertions, the original node map has 100,000 entries and the compact map has zero. The index-only substitution retains the same history, supporting the distinction between index cost and object lifetime. Destroying the queue reduces traced residual allocations to roughly 1.5 KiB for the original and 2.1 KiB for compact, including bookkeeping. The retained state belongs to the lifetime of the queue.

5. Runtime results and baseline choice

Before comparing learned queues with direct heapq operations, we measure the shared queue wrapper itself. Its median CPU cost is 1.11-1.18 times the direct heapq baseline in the same distance-to-vertex routing kernel. This control separates wrapper dispatch from the larger costs reported below; native controls later test the same question with C++ queues.

Base graph	Original / heap	Bitmap / heap	Compact / heap
Brussels	11.40	12.25	11.72
New York	9.93	10.28	8.75
Grid 2,025	12.21	12.65	3.04
Grid 8,100	12.74	13.18	2.52
Geometric 2,048	8.79	9.32	9.05
Geometric 8,192	7.68	8.09	7.95
Sparse 2,048	9.00	9.25	4.71
Sparse 8,192	8.85	8.88	2.58

Table 3. Python median source-level CPU-time ratios against the direct heapq operations in the common routing-map kernel. All learned variants shown use the same linear advisor. Values above one mean slower.

Original + linear costs 7.68 to 12.74 times the shared-routing heapq baseline across the eight base graphs. Bitmap/original linear-advisor ratios range from 1.001 to 1.069; sparse vEB/original ratios range from 1.045 to 1.134. These index-only replacements provide no graph-level median improvement on the eight base graphs. Small shifted-case improvements also occur: sparse vEB/original point estimates are approximately 0.991 on the lopsided target and 0.992 on the sparse x4 target.

The combined compact repair has large benefits on integer grids. Relative to the index-only bitmap queue, its paired CPU ratio is 0.242 [0.239, 0.257] on the 2,025-node grid and 0.192 [0.178, 0.204] on the 8,100-node grid. These exploratory intervals collapse repetitions before resampling three training-source clusters. Compact still costs 3.04 and 2.52 times the heapq baseline respectively. Because compact changes grouping, bucket removal, node lifetime, and hint/height history together, these gains do not by themselves identify one repair component; Section 6 tests two narrower changes.

A separate predecessor microbenchmark replays 10,000-operation traces containing 25% insertion, 25% removal, and 50% strict-predecessor queries. Sixteen universe/cardinality/distribution cases receive five shuffled repetitions and separate traced-memory runs. Insertions are guaranteed absent, avoiding an extra membership check for SortedList. Bitmap/SortedList CPU ratios range from 0.924 to 2.490, with lower point estimates in two cases; sparse vEB ratios range from 1.069 to 2.173. The microbenchmark isolates predecessor-index costs.

The native follow-up was designed after the Python findings and frozen before its own measurements. It uses grouped live-key skip lists with common bookkeeping across plain and guided variants, std::set or bitmap rank indexes, exact/linear/histogram advisors, multiset and integer-radix controls. It also compares three heaps: a direct (distance, reverse-insertion-sequence, vertex) heap; a scalar heap with the common routing map; and a scalar heap grouping duplicate priorities. The reverse sequence preserves LIFO ties. Selftests compare exact inserted-key sequences and insertion/extraction/peak counts between direct and shared heaps.

Native target	Plain / direct	Guided / direct	Grouped heap / direct	Shared heap / direct
Brussels	10.10	10.63	5.09	3.41
Geometric 8,192	8.95	8.98	4.66	3.33
Grid 8,100	1.43	1.64	1.19	1.70
New York	6.31	6.37	3.38	2.59
Sparse 2,048 lopsided	7.02	7.27	3.75	2.63
Sparse 2,048 x4	3.44	3.61	2.20	2.22
Sparse 8,192	1.48	1.64	1.11	1.59

Table 4. First-host (Mac) native CPU ratios to the direct heap. Plain denotes grouped skip list; guided denotes bitmap with linear prediction. Six query/model cases per target each collapse seven paired repetitions.

All 42 first-host native cases and 3,444 timed method records pass distance and drain checks; full binary64 distance outputs match independent Python references. Optimized and AddressSanitizer and UndefinedBehaviorSanitizer selftests also pass. Queue construction/destruction and ordinary runtime guards are timed; training, oracle comparisons, checksums, and diagnostic validation are outside the timer. All fitted models remain resident.

On the Mac, the direct heap has the lowest graph-level median among all tested native methods on every target. Guided bitmap + linear costs 1.64 to 10.63 times the direct heap. Bitmap/set linear-advisor ratios range from 0.800 to 0.995, but the lopsided case's exploratory interval includes parity. Guided bitmap + linear/plain grouped-skiplist ratios range from 1.008 to 1.185, with the New York interval also including parity. All six guided variants have slower graph-level point estimates than the plain grouped skip list. An improved predecessor index therefore does not demonstrate a benefit from advice itself in this implementation.

On the Mac, baseline interface choice materially changes the apparent result. The shared scalar heap costs 1.59 to 3.41 times the direct heap across native targets. Duplicate grouping improves the shared heap on the grid and large sparse graph, with grouped/shared paired CPU ratios of 0.708 and 0.703, despite using no advice. The direct heap still wins those graph-level comparisons. These controls support reporting both a shared-kernel ablation baseline and a strong deployable baseline. Entry representation and associated work differ between the two interfaces.

For a measured training/build cost T and a positive paired query saving H-A, a hypothetical cold model breaks even only when the number of repeated queries exceeds T/(H-A). We charge one collecting training search plus the selected

model construction. None of the 42 held-out query/model cases has positive median paired savings for native bitmap + linear against the direct heap on either clock, so no finite cold break-even is observed for that comparison.

6. Controlled bookkeeping ablations

To separate two components of the combined repair, we cross rank-bucket representation (original list or a linked container with one entry per queued item) with node-map ownership (retain history or erase the mapping after its last live occurrence). All four variants use the same bitmap predecessor index, original skip-list methods, one node per insertion, and unchanged value-to-rank occurrence lists. The linked container preserves append order, tail hints, and first-equal removal with a value-to-deque handle map. Its expected constant-time removal includes additional hashing and allocation costs. Map cleanup leaves node pointers intact. The compact queue is a fifth, explicitly coupled reference.

The implementation rationale for last-occurrence cleanup is that the only node-map read used for hint selection takes its key from a nonempty live rank bucket. Provided rank/value occurrence bookkeeping remains consistent, a key with no live occurrence cannot be read through that path. Reinsertion assigns a new node mapping before the key becomes a subsequent hint. Removing an exhausted mapping therefore preserves these reads without requiring immediate reclamation of the node. Section 8 gives a counterexample for pointer detachment on a removed duplicate that still represents a live key.

Before timing, 3,600 bucket mutations and 288 counted trace comparisons passed the frozen semantic preflight; 576 counted/counter-free ungrouped-queue executions, 72 compact traces, and 60 small graph checks also passed. A separate full-workload gate ran all four ungrouped variants on all 24 query/model cases using the first planned height seed per case. Its 96 executions matched exact distances and six event streams: queue operations, comparison operands, hint identities, height draws, node creation, and next-pointer assignments. The four variants preserve these streams on every tested query/model case.

The post-hoc mechanism study uses the 8,100-node grid, 8,192-node geometric graph, New York, and the lopsided sparse target, each with the existing three training sources and near/far queries. It fixes linear advice and retains base-graph training for the shifted target. Five paired repeats per small case and three per New York case yield 108 fresh-process blocks and 540 timings. Every method has a checked warmup and fresh timed queue with matched outside-timer seeds; order is shuffled in the frozen plan. Training, validation, and forced cleanup remain outside query clocks. All 178 planned workers (108 timing, 40 graph-memory, 30 bounded-live memory) completed with matching specifications and source hashes. All timing distance/drain checks passed. 27 timings have wall/CPU ratios above 1.10 and 15 above 1.25; none is excluded.

Target	Cleanup / history, list	Cleanup / history, linked	Linked / list, history	Linked / list, cleanup
Grid 8,100	1.007 [1.007, 1.009]	1.017 [1.007, 1.027]	1.098 [1.082, 1.104]	1.107 [1.088, 1.154]
Geometric 8,192	1.017 [1.009, 1.021]	1.011 [0.994, 1.017]	1.181 [1.176, 1.185]	1.176 [1.170, 1.180]
New York	1.007 [0.987, 1.020]	1.002 [0.940, 1.009]	1.167 [1.160, 1.206]	1.159 [1.122, 1.167]
Sparse 2,048 lopsided	1.008 [1.007, 1.011]	1.013 [1.009, 1.018]	0.777 [0.770, 0.789]	0.779 [0.777, 0.783]

Table 5. Conditional CPU-time ratios in the two-by-two bookkeeping design. Values below one favor the numerator. Brackets are exploratory 95% source-cluster percentile intervals, based on only three training clusters. Ratios are formed within blocks, repetitions collapse within source, then the six query/model cases form each graph median. The substitutions include allocation and hash costs.

Target	Interaction, CPU	Compact / linked-cleanup, CPU	Compact / linked-cleanup, wall
Grid 8,100	1.003 [1.001, 1.012]	0.166 [0.157, 0.177]	0.166 [0.157, 0.175]
Geometric 8,192	0.997 [0.994, 1.002]	0.820 [0.813, 0.836]	0.820 [0.813, 0.839]
New York	0.987 [0.950, 0.998]	0.707 [0.707, 0.736]	0.707 [0.706, 0.744]
Sparse 2,048 lopsided	1.002 [0.980, 1.010]	0.874 [0.872, 0.878]	0.872 [0.868, 0.879]

Table 6. Interaction is (linked-cleanup/list-cleanup)/(linked-history/list-history), calculated inside each paired block before aggregation. Parity denotes no multiplicative interaction in this contrast. The comparison with the combined compact implementation changes grouping, per-value metadata, hint/height history and detachment together. All wall-clock factorial contrasts and case-level values are included in the artifact.

Map cleanup has a small query-time cost in the observed CPU medians: the eight conditional ratios range from 1.002 to 1.017, with several exploratory intervals crossing parity. The linked bucket has a different pattern: its CPU ratios are approximately 0.777-0.779 on the lopsided target (about 22% lower time), but 1.098-1.181 on the grid, geometric, and New York targets (about 10-18% higher time). The original value-to-rank lists remain in all four arms.

The multiplicative interaction medians are close to parity (0.987-1.003). The comparison with the combined compact implementation remains substantial on the grid: CPU ratio 0.166 [0.157, 0.177] against linked buckets with map cleanup. Thus the two isolated changes do not account for the earlier combined grid speedup. The remaining bundle includes grouping, per-value metadata, hint/height history, and detachment. Wall-clock medians agree with the broad CPU direction of these contrasts.

Memory uses one fixed training/far-query case per target and method, in separate processes with automatic cyclic GC either enabled or disabled during the query. Immediately after querying, tracing captures the current allocation and peak; distance/routing outputs are then dropped while the queue and advisor remain alive. Automatic collection stays disabled until the explicit before/after-GC snapshots are taken. Queue statistics are allocated only afterward. These boundaries distinguish map cardinality, reachable storage, and delayed collection.

New York variant	Query GC	Query peak MiB	Fitted peak MiB	Empty pre-GC MiB	Empty post-GC MiB
List + history	On	102.379	102.379	100.362	100.362
List + history	Off	102.380	102.380	100.363	100.362
List + cleanup	On	10.151	13.619	0.058	0.053
List + cleanup	Off	10.151	13.619	0.058	0.053
Linked + history	On	102.389	102.389	100.368	100.363
Linked + history	Off	102.390	102.390	100.369	100.363
Linked + cleanup	On	10.282	13.619	0.064	0.054
Linked + cleanup	Off	10.282	13.619	0.064	0.054
Compact (coupled)	On	10.153	13.619	0.054	0.049
Compact (coupled)	Off	10.153	13.619	0.054	0.049

Table 7. Controlled New York memory measurements. Empty-queue snapshots retain the advisor but release distance/routing outputs. Fitted peak is max(training/build, query peak). Graph/oracle, prewarmed libraries, and untracked native memory are excluded. Each row is one fixed-source worker.

With ordinary automatic GC in the fixed New York case, map cleanup alone reduces the list-bucket query peak from 102.379 to 10.151 MiB (10.09-fold). The combined compact implementation uses 10.153 MiB, so its grouping and other policy changes are unnecessary for nearly all of this measured memory reduction. Accounting for the 13.619 MiB training/build peak gives a 7.52-fold reduction in the traced fitted-run peak. Both map-cleaned arms have zero retained node-map entries after draining; both historical arms retain 255,777. The same large memory reduction remains when automatic GC is disabled during the query.

The faster linked container also has a storage cost. On the fixed lopsided memory case with automatic GC enabled and map cleanup, its query peak is 2.505 MiB versus 1.258 MiB for the list container. Thus the observed speed benefit on that target comes with roughly twice the query-phase traced allocation. The full graph/method/GC memory table is included in the artifact.

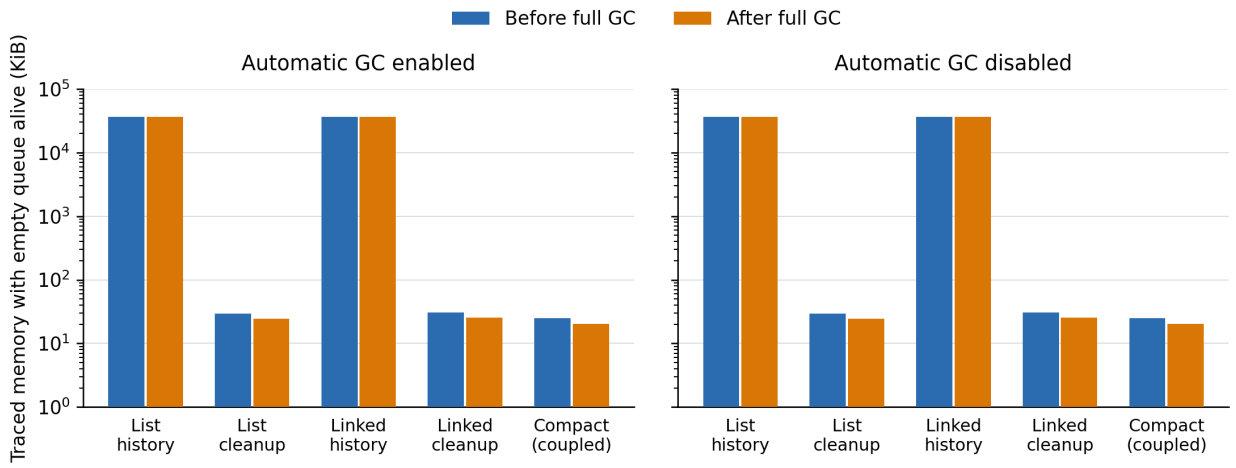


Figure 2. Bounded-live memory after 100,000 unique insertions with at most 128 entries live, constant rank zero, and a drained queue still alive. Left: automatic cyclic GC enabled during the trace; right: disabled. Both panels show measurements before and after explicit full GC on a logarithmic scale. No model or graph is present, and unique priorities make duplicate grouping inactive. Results at 1,000 and 10,000 insertions and all raw records are also preserved.

In the 100,000-insertion bounded-live test, list-bucket history retains 35.183 MiB after full GC, compared with 0.02386 MiB for map cleanup alone. The node-map counts are 100,000 and zero respectively. With automatic GC disabled, the cleaned list queue already retains only 29.437 KiB before explicit collection. Thus delayed cyclic collection does not explain the bulk difference on this trace. The linked-cleanup arm retains 25.462 KiB and compact retains 20.404 KiB after GC. These unique-key traces do not activate duplicate grouping. Destruction leaves roughly 1.2-1.3 KiB of traced report/runtime allocations in all five variants, supporting an ownership-within-queue-lifetime interpretation. Cleaned retention is nearly flat over the measured lengths.

Across all 70 memory workers, explicit full collection while the empty queue remains alive reports zero collected objects. Enabling versus disabling automatic cyclic GC changes graph-query peaks by at most 600 bytes and bounded-live total peaks by at most 656 bytes. The few-KiB before/after-collection differences are therefore not evidence of reclaimed cycles; full collection also affects interpreter free lists. Under these traces, historical references account for the large retention effect, and delayed cyclic collection has no material measured influence.

7. Second-host replication

We repeated the full native protocol and the bounded-live Python retention experiment on a second physical laptop: a Dell Latitude 5400 with an Intel Core i5-8365U (four cores, eight threads), 8 GiB installed RAM, and Windows 11 Pro 25H2 (build 26200.9168). Execution used Ubuntu 24.04 under WSL2, with about 3.71 GiB visible RAM, 1 GiB swap, and eight visible logical CPUs. Recorded before/after snapshots showed zero swap use; per-case peak RSS was not measured. The native build used g++ 13.3.0, C++17, -O3 and -DNDEBUG without fast-math. Python was 3.12.3, with the same NumPy, NetworkX, and SortedContainers versions as the first host.

The native rerun preserved the seven targets, 42 training/query cases, seven repetitions, adjacency/input hashes, and numeric height seeds. All 3,444 new timings passed distance and drain checks; optimized and ASAN/UBSAN selftests also passed. The first-host and second-host results were aggregated separately from raw CSVs using the same within-repetition ratios and source collapse. Actual randomized method permutations differed between the builds and are archived. Original experimental sources and observations were preserved.

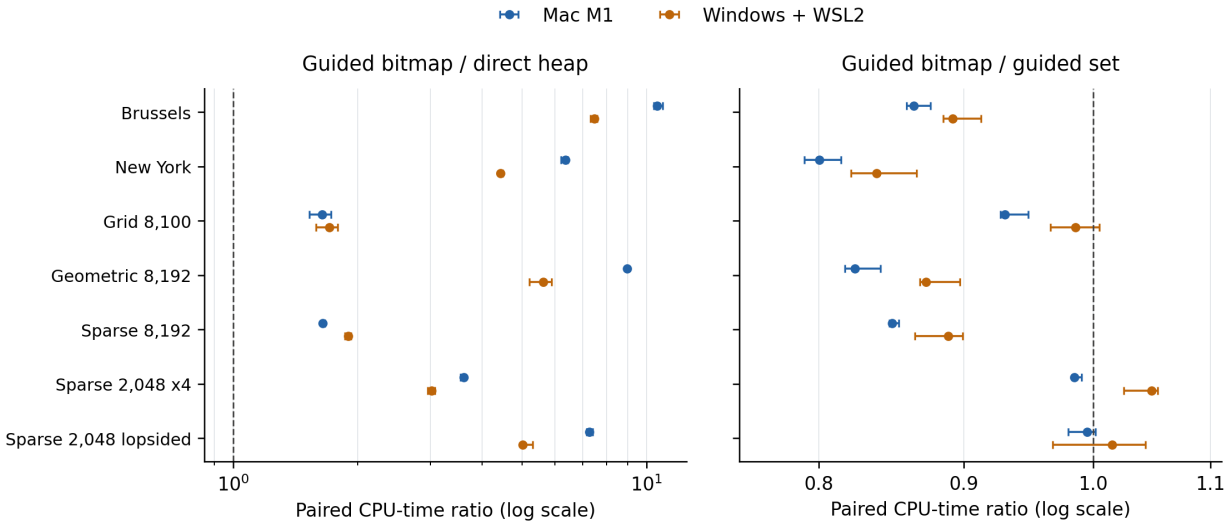


Figure 3. Native CPU-time ratios with linear advice on the two hosts. Left: the direct application heap remains faster on all seven targets. Right: the bitmap-versus-set index comparison varies by host. Points are medians of six case-level paired ratios; bars are exploratory 95% intervals resampling three training-source clusters. Hosts are summarized separately. The vertical line marks parity.

The direct heap has the lowest graph-level median on every target on both hosts, for CPU and wall clocks. Guided bitmap + linear costs 1.71-7.48 times the direct heap on Windows + WSL2, compared with 1.64-10.63 on the Mac. All 42 workload/advisor/index guided-to-plain-skip-list median ratios exceed one on each host and clock. The deployment-baseline conclusion therefore persists across these configurations, while 28 of the 82 descriptive wall-clock method rank positions change.

Finer implementation effects vary. On the scaled sparse target, bitmap/set linear-advisor CPU ratios change from 0.985 [0.983, 0.990] on the Mac to 1.049 [1.025, 1.054] on Windows + WSL2. Exact and histogram advisors also reverse direction on that target. Index improvement in one configuration is therefore insufficient to predict the other configuration.

The common-routing heap further illustrates baseline sensitivity. On the grid, guided bitmap + linear/shared-heap CPU ratio is 0.978 [0.913, 1.013] on the Mac, near parity with an interval spanning one, versus 1.235 [1.210, 1.264] on Windows + WSL2. Grouped/shared-heap CPU point ratios also change from 0.708 to 1.099 on the grid and from 0.703 to 1.027 on the large sparse graph. The direct heap remains faster in all of these comparisons.

Variant	Mac retained MiB	Windows + WSL2 retained MiB	Node-map entries (both)
List + history	35.18298	34.41990	100,000
List + cleanup	0.02386	0.02375	0
Linked + history	35.18399	34.42091	100,000
Linked + cleanup	0.02487	0.02476	0
Compact (coupled)	0.01993	0.01987	0

Table 8. Second-host retention comparison after 100,000 unique insertions, at most 128 live entries, constant rank zero, seed 101, automatic GC enabled, and a drained queue alive after full GC. These are Python-traced allocations, including small runtime state. Node-map entries count dictionary keys, not all reachable objects.

All 30 second-host retention workers passed the unchanged trace specs and extraction checks in fresh sequential processes. On Windows + WSL2, list-bucket history retains 34.420 MiB after full GC, compared with 0.02375 MiB for map cleanup alone. Retained node-map entries are 100,000 and zero, matching the Mac. The ownership mechanism and cleanup benefit persist across the two Python versions and platforms. Absolute traced bytes differ, as expected for distinct interpreters and runtimes.

Automatic-GC modes give the same post-full-GC retention on both hosts, and every explicit collection in the bounded-live tests reports zero collected objects. Destruction leaves about 1.2-1.3 KiB of traced report/runtime allocations in each arm. The rerun covers native timings and bounded-live ownership; the Python graph timings, GC graph-memory cases, and per-insertion timing ablations remain first-host experiments.

8. Interpretation and limitations

Historical ownership explains the large retention cost: map cleanup removes almost all of the excess storage while preserving the tested queue execution streams. Its small runtime cost contrasts with the workload-dependent tradeoff of linked buckets. The native controls further show that auxiliary-index improvements can be outweighed by other queue operations and the application interface.

The native and bounded-live experiments now cover two host configurations; the Python graph timings and graph-memory cases remain on the Mac. Each graph still has three training sources and one synthetic instance per family/size, and transformed targets reuse those inputs. Source-cluster intervals are exploratory. Hardware, OS, compiler/standard library, allocator, and Python version change together across hosts, so differences identify configuration sensitivity rather than an isolated architecture effect. WSL2 is a virtualized Linux environment with recorded memory limits. Actual method permutations differ across builds despite matched numeric seeds; ratios are paired within host, and hosts are not pooled as additional graph samples. Numeric keys and fixed monotone advisors define the benchmark contract. The original paper supplies guarantees for its advice/cost models and reports comparison-count experiments; our results address the declared implementations.

Attribution follows the intervention boundaries. The four ungrouped variants isolate map ownership and a concrete bucket substitution; the compact control also changes grouping, hint representatives, height draws, and pointer lifetime. Its residual gain therefore belongs to that bundle. Full event-stream parity covers one planned height seed per source case, alongside distance/drain checks for every timing. The ablation targets were chosen after the earlier results and privately frozen before execution. Memory figures measure traced allocations for the stated phases and retained objects; they exclude graph/oracle loading and untracked native memory. Nearly flat cleaned retention is not a constant-space proof, since retained capacities and maximum sampled height can vary. Scale interventions sometimes change Python integer weights into floats, so cross-target effects also include numeric representation.

A rejected cleanup attempt exposes a representative dependency. With identity rank predictions and height seed 101, insert priority 1 twice, extract one minimum, and then insert priority 2. The original occurrence queue and last-occurrence map-cleanup controls produce the expected full extraction sequence 1, 1, 2; blindly clearing the extracted node's links raises IndexError. The newest equal-key node can be extracted while another occurrence remains, leaving sl.nodes[1] pointing to the extracted node as a later hint. Clearing those links breaks our modification. Safe detachment therefore requires an additional representative policy.

Before reporting the finding upstream, a bounded primary-source and repository search found no previous matching report of this exact retention mechanism. We have now filed the runnable reproducer and candidate fix as repository issue #1 [12]. Duplicate grouping, bitmap/vEB indexing, and practical queue evaluation are established methods [1, 3, 4, 6]. The contribution is the ownership mechanism, the hint dependency that constrains cleanup, and the controlled costs of the tested interventions.

9. Conclusion

The released queue retains metadata for historical keys, yielding substantial traced-memory growth despite a bounded live queue. Maintainer fix: delete the sl.nodes entry for a key when its last live occurrence is extracted; node pointers can be left intact (Section 8). Here a live occurrence is one copy of the priority that is still queued. This cleanup removes nearly all of the retained-storage cost on both tested hosts while preserving the validated search behavior. Its small query-time cost and the workload-dependent bucket tradeoff are measured in the Mac graph ablation. The direct heap remains the strongest native baseline across the two configurations, even as index and shared-interface comparisons change direction. These results support explicit accounting for historical ownership, cleanup semantics, prediction/index costs, and application interfaces.

Artifact availability

The accompanying artifact contains source, raw observations, input hashes, frozen computational plans, correctness checks, and reproduction commands. Upstream source and road data are fetched separately by pinned hashes. The feasibility pilot is excluded from reported counts. AI coding assistants were used as tools for code generation, debugging, and editing of prose. The author designed the experiments, ran all measurements on both hosts, verified all reported figures against the raw artifact, and is responsible for all claims.

References

[1] Z. Benomar and C. Coester. Learning-Augmented Priority Queues. NeurIPS, 2024. [link]

[2] Z. Benomar. Learning-augmented-priority-queues, released source; pinned commit 3915cc261e87949b2b0aa724673c2227492d1b08. [link]

[3] M. Chen, R. A. Chowdhury, V. Ramachandran, D. L. Roche, and L. Tong. Priority Queues and Dijkstra's Algorithm. University of Texas at Austin, Technical Report TR-07-54, 2007. [link]

[4] P. Dinklage, J. Fischer, and A. Herlez. Engineering Predecessor Data Structures for Dynamic Integer Sets. SEA, 2021. [link]

[5] R. K. Ahuja, K. Mehlhorn, J. B. Orlin, and R. E. Tarjan. Faster Algorithms for the Shortest Path Problem. Journal of the ACM, 1990. [link]

[6] D. H. Larkin, S. Sen, and R. E. Tarjan. A Back-to-Basics Empirical Study of Priority Queues. ALENEX, 2014. [link]

[7] A. Kipf, R. Marcus, A. van Renen, M. Stoian, A. Kemper, T. Kraska, and T. Neumann. SOSD: A Benchmark for Learned Indexes. NeurIPS Workshop on Machine Learning for Systems, 2019. [link]

[8] Q. Liu, S. Han, Y. Qi, J. Peng, J. Li, L. Lin, and L. Chen. Why Are Learned Indexes So Effective but Sometimes Ineffective? PVLDB 18, 2025. [link]

[9] L. Bellomo, G. Cianci, L. de Rosa, P. Ferragina, and M. Odorisio. A Comparative Study of Compressed, Learned, and Traditional Indexing Methods for Integer Data. SEA, 2025. [link]

[10] T. Lykouris and S. Vassilvitskii. Competitive Caching with Machine Learned Advice. ICML, PMLR 80:3296-3305, 2018. [link]

[11] M. Mitzenmacher and S. Vassilvitskii. Algorithms with Predictions. In Beyond the Worst-Case Analysis of Algorithms, Chapter 30, pages 646-662, Cambridge University Press, 2021. [link]

[12] S. Siddiqui. OnlineSL retains historical sl.nodes entries after the final occurrence is extracted. GitHub issue #1, opened 6 November 2026. [link]